

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

федеральное государственное автономное образовательное
учреждение высшего образования



**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Инженерная школа информационных технологий и робототехники

09.04.04 «Программная инженерия»

Отделение информационных технологий

Отчет по лабораторной работе №1

по дисциплине «Численные методы решения прикладных задач»

Выполнил:

Студент группы 8МП5Л

_____ Шаврин А.П.

Проверил:

К.х.н., доцент (ОНД, ИШПР)

_____ Зарубин А.Г.

Цель

Изучить методику построения экспоненциальной регрессионной модели по экспериментальным данным вибрации электродвигателя магистрального насосного агрегата (МНА), освоить процедуру подбора параметров нелинейной модели методом наименьших квадратов и оценить точность прогноза на отложенной (тестовой) выборке.

Задание

1. Загрузить обучающую (train.csv) и тестовую (test.csv) выборки, содержащие значения вибрации v (мм/с) в зависимости от времени наработки t (ч).
2. Рассчитать начальные приближения параметров экспоненциальной модели
3. Выполнить подгонку параметров модели a и b методом градиентного спуска (аналог `nls()` в R - `scipy.optimize.curve_fit` в Python).
4. Рассчитать прогнозные значения на тестовой выборке
5. Оценить точность прогноза метриками MAE, MSE, RMSE, MAPE
6. Построить график исходных данных и полученной модели

Выполнение работы

1. В качестве модели принята стандартная экспоненциальная зависимость:

$$v(t) = a * e^{(b * t)}$$

2. Начальные приближения рассчитаны по формулам:

$a_{start} = v_1 = 1.5$ - первая точка обучающей выборки (при $t_1 = 0$)

$$b_{start} = \frac{\ln(v_n / v_1)}{(t_n - t_1)} \approx 9.305 \cdot 10^{-5}$$

, где $v_1 = 1.5$, $v_n = 4.8$, $t_1 = 0$, $t_n = 12500$.

Загрузка тренировочных и тестовых данных, настройка модели и получение начальных приближения в коде приведено на рисунке 1.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy.optimize import curve_fit
5
6 # Загрузка данных
7 train = pd.read_csv('train.csv', sep=';', decimal=',')
8 test = pd.read_csv('test.csv', sep=';', decimal=',')
9
10 t_train = train['t'].values.astype(float)
11 v_train = train['v'].values.astype(float)
12 t_test = test['t'].values.astype(float)
13 v_test = test['v'].values.astype(float)
14
15 # Модель
16 def exp_model(t, a, b):
17     """v(t) = a * exp(b * t)"""
18     return a * np.exp(b * t)
19
20 # Начальные приближения
21 a_start = v_train[0]
22 b_start = np.log(v_train[-1] / v_train[0]) / (t_train[-1] - t_train[0])
23
24 print(f"a_start = {a_start}")
25 print(f"b_start = {b_start:.10e}")
```

Problems 1 Output Debug Console Terminal Ports

```
~/Doc/D/LetiMaster/3_semester/NumericalMethodsForSolvingAppliedProblems/lab1
uv run python main.py
a_start = 1.5
b_start = 9.3052064784e-05
```

Рисунок 1. Получение начальных приближения.

3. Подгонка параметров модели выполнена методом нелинейного МНК (`scipy.optimize.curve_fit`, метод Левенберга–Марквардта — идейный аналог `nls()` в R) и представлена на рисунке 2.

```
27 # Подгонка модели
28 popt, pcov = curve_fit(exp_model, t_train, v_train, p0=[a_start, b_start])
29 a_model, b_model = popt
30
31 # Стандартные ошибки
32 perr = np.sqrt(np.diag(pcov))
33
34 print("\n=== ПАРАМЕТРЫ МОДЕЛИ ===")
35 print(f"a = {a_model:.6f} (std err = {perr[0]:.6f})")
36 print(f"b = {b_model:.10e} (std err = {perr[1]:.10e})")
37
38 # Прогноз на тестовой выборке
```

Problems 1 Output Debug Console Terminal Ports

```
=== ПАРАМЕТРЫ МОДЕЛИ ===
a = 1.209456 (std err = 0.057790)
b = 1.0860896110e-04 (std err = 5.0206785984e-06)
```

Рисунок 2. Подгонка параметров модели.

4. Прогноз на тестовой выборке приведен на рисунке 3.

```
38 # Прогноз на тестовой выборке
39 v_pred_test = exp_model(t_test, a_model, b_model)
40 test['pred'] = v_pred_test
41
42 print("\n=== ПРОГНОЗ НА ТЕСТЕ ===")
43 print(test.to_string(index=False))
44
```

Problems 1 Output Debug Console Terminal Ports

```
=== ПРОГНОЗ НА ТЕСТЕ ===
   t    v    pred
13200 5.80 5.072375
14000 6.75 5.532813
```

Рисунок 3. Прогноз модели на тестовой выборке.

5. Расчет метрик точности для тестовой выборки приведен на рисунке 4.

```

45 # Метрики
46 error = v_test - v_pred_test
47 MAE = np.mean(np.abs(error))
48 MSE = np.mean(error ** 2)
49 RMSE = np.sqrt(MSE)
50 MAPE = np.mean(np.abs(error / v_test)) * 100
51
52 print("\n=== МЕТРИКИ НА ТЕСТЕ ===")
53 print(f"MAE = {MAE:.4f}")
54 print(f"MSE = {MSE:.4f}")
55 print(f"RMSE = {RMSE:.4f}")
56 print(f"MAPE = {MAPE:.2f}%")
57

```

Problems 1 Output Debug Console Terminal Ports

```

=== МЕТРИКИ НА ТЕСТЕ ===
MAE = 0.9724
MSE = 1.0055
RMSE = 1.0027
MAPE = 15.29%

```

Рисунок 4. Расчет метрик точности модели на тестовой выборке.

6. График полученной модели представлен на рисунке 5.

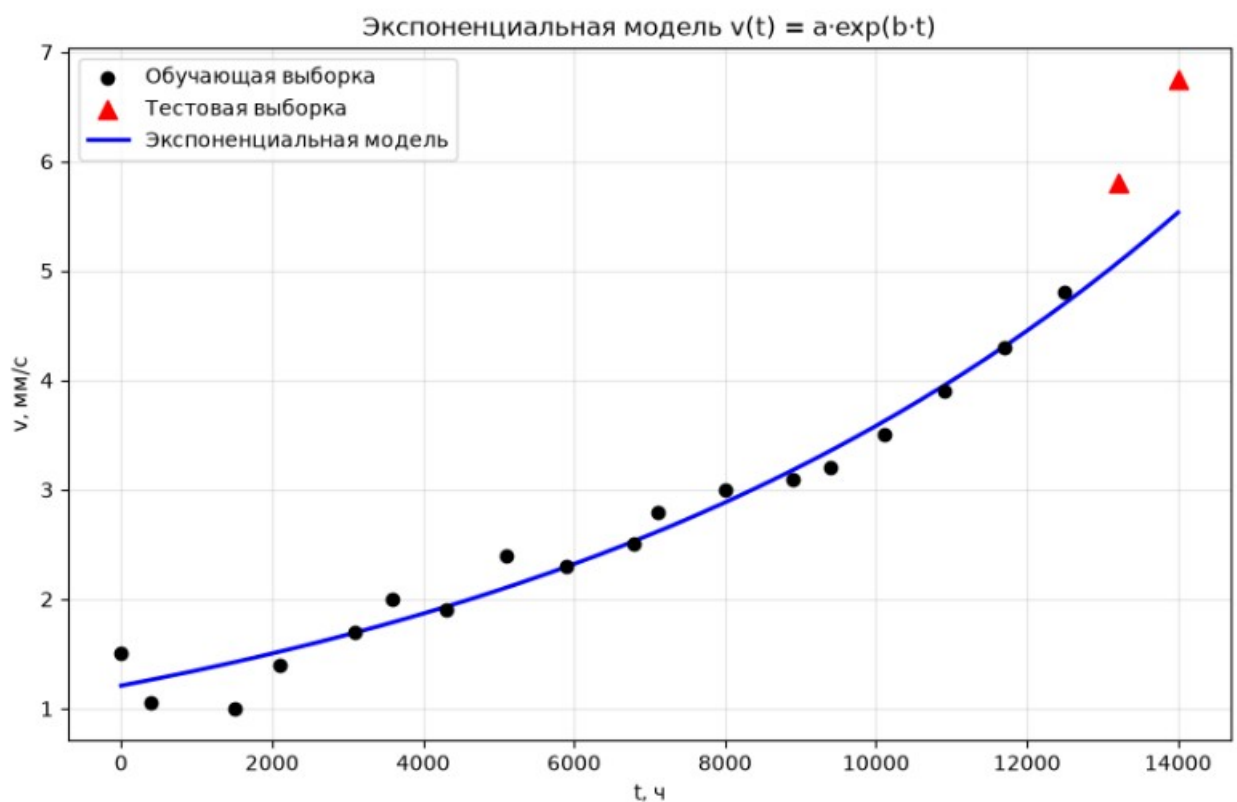


Рисунок 5. График модели.

Вывод

В ходе лабораторной работы была построена экспоненциальная модель зависимости вибрации электродвигателя МНА от времени наработки:

$$v(t) = 1.209456 * e^{1.086 * 10^{-4} * t}$$

Основные результаты:

1. Методом нелинейного МНК получены оценки параметров модели: начальный уровень вибрации 1.2095 мм/с и скорость роста $1.086 \cdot 10^{-4}$ 1/ч.
2. На тестовой выборке (точки при 13200 и 14000 ч) модель дала прогноз 5.07 и 5.53 мм/с против фактических 5.80 и 6.75 мм/с. Средняя абсолютная процентная ошибка MAPE составила 15.29 % — точность удовлетворительная, но невысокая.

Таким образом, цель работы достигнута: освоена методика построения экспоненциальной модели, выполнена оценка её параметров и точности

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

main.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Загрузка данных
train = pd.read_csv('train.csv', sep=';', decimal=',')
test = pd.read_csv('test.csv', sep=';', decimal=',')

t_train = train['t'].values.astype(float)
v_train = train['v'].values.astype(float)
t_test = test['t'].values.astype(float)
v_test = test['v'].values.astype(float)

# Модель
def exp_model(t, a, b):
    """v(t) = a * exp(b * t)"""
    return a * np.exp(b * t)

# Начальные приближения
a_start = v_train[0]
b_start = np.log(v_train[-1] / v_train[0]) / (t_train[-1] - t_train[0])

print(f"a_start = {a_start}")
print(f"b_start = {b_start:.10e}")

# Подгонка модели
popt, pcov = curve_fit(exp_model, t_train, v_train, p0=[a_start, b_start])
a_model, b_model = pop

# Стандартные ошибки
perr = np.sqrt(np.diag(pcov))

print("\n=== ПАРАМЕТРЫ МОДЕЛИ ===")
print(f"a = {a_model:.6f} (std err = {perr[0]:.6f})")
print(f"b = {b_model:.10e} (std err = {perr[1]:.10e})")

# Прогноз на тестовой выборке
v_pred_test = exp_model(t_test, a_model, b_model)
test['pred'] = v_pred_test

print("\n=== ПРОГНОЗ НА ТЕСТЕ ===")
print(test.to_string(index=False))

# Метрики
error = v_test - v_pred_test
MAE = np.mean(np.abs(error))
MSE = np.mean(error ** 2)
RMSE = np.sqrt(MSE)
MAPE = np.mean(np.abs(error / v_test)) * 100

print("\n=== МЕТРИКИ НА ТЕСТЕ ===")
```

```

print(f"MAE = {MAE:.4f}")
print(f"MSE = {MSE:.4f}")
print(f"RMSE = {RMSE:.4f}")
print(f"MAPE = {MAPE:.2f}%")

# График
t_grid = np.linspace(t_train.min(), t_test.max(), 300)
v_grid = exp_model(t_grid, a_model, b_model)

plt.figure(figsize=(10, 6))
plt.scatter(t_train, v_train, color='black', label='Обучающая выборка', zorder=3)
plt.scatter(t_test, v_test, color='red', marker='^', s=80,
            label='Тестовая выборка', zorder=3)
plt.plot(t_grid, v_grid, color='blue', linewidth=2,
         label='Экспоненциальная модель')
plt.xlabel('t, ч')
plt.ylabel('v, мм/с')
plt.title('Экспоненциальная модель  $v(t) = a \cdot \exp(b \cdot t)$ ')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('plot.png')
plt.show()

# Итоговая модель
print(f"\nИтоговая модель:  $v(t) = \{a\_model:.6f\} * \exp(\{b\_model:.8e\} * t)$ ")
print(f"MAPE на тесте: {MAPE:.2f}%")

```